Code Anaylsis:

a. Name of two GenAI programs: Claude and Google Gemini

b. How you accessed them (e.g., Google query, CoPilot on my computer)
Claude was accessed through the Claude website on my computer and Gemini was accessed through the Google Gemini website on my computer.

c. Prompt used. (You must use the same prompt for both GenAIs.)

generate C++ code using objects, not functions, to implement the following requirements :

- Sudoku puzzles s are usually constructed to have a unique solution. This

requires that at least 17 cells be filled in (and may require more; depending on the puzzle and which cells are empty, as many as 78 filled cells can be required to guarantee uniqueness). In general, the number of cells that are already filled in is not a particularly good indicator of how difficult a Sudoku puzzle is to solve

-The Sudoku problem, then, is this: Given a partially completed Sudoku puzzle, find the (or at least a) solution; if no solution exists, prove that.
• It is possible to solve a Sudoku puzzle entirely through logic. For example, in the above puzzle, consider the bottom right block, top right cell. Because of other numbers in the same row, column, or grid, that number cannot be 1, 2, 3, 5, 6, 7, 8, or 9, and so must be 4. By proceeding in this fashion, using these and more advanced deduction rules, it's possible to gradually fill in the entire grid.
• That is not the approach we're going to use. Instead, we're going to implement a brute-force search, in which we exhaustively try possibilities until we find a solution. This will be slower to run but is easier to solve. (And by "slower to run," we mean it'll finish in a few tenths of a second rather than a few milliseconds.)

-Suppose we have a Sudoku puzzle of which k of the 81 cells are unfilled. We identify an empty cell and deduce a list of candidates for what might go into that cell. If there are no such values (all digits are accounted for elsewhere in the same row, column, or grid), then we must have made a mistake in filling in the k cells and return a no-solution-found result. Otherwise, we try putting the first candidate value into the empty cell, and see if we can find a solution with the k+1 cells filled (i.e. a recursive call). If we

get a no-solution-found to the first candidate, then we try the next, and so on, until we either find a solution (k == 81) or determine that no solution exists (all candidates report no solution, which again implies we took a wrong path somewhere earlier).
• This approach, by the way, is called depth-first search with backtracking. For each empty cell we make an attempt and see how far we can get with it. If we fail to find a solution, we back up until we find something else to try, and so on. If a solution exists, we are guaranteed to find it, because we exhaustively try all possibilities.
• You are given five files (puzzle1.txt through puzzle5.txt), each containing a Sudoku puzzle. Digits already filled in have been placed, and an underscore character (_) is used to indicate a blank square.
• Write a recursive object-oriented program to find the solution to the five puzzle files.
• The output for each solution, should include:
o The puzzle file name (e.g., puzzle1.txt)
o The puzzle from the file printed out.
o The solution to the puzzle printed out or if no solution is found, “No solution found” printed out.
• The solution may not be unique; if there is more than one solution to the puzzle, your program should print them all out.

*Keep the code organized, include comments for every line

d. Generated code from both GenAIs.
Claude:

```
// sudoku.cpp - Object-oriented recursive (backtracking) Sudoku solver
// Finds and prints ALL solutions for each of puzzle1.txt ... puzzle5.txt

#include <iostream>   // for std::cout, std::cerr
#include <fstream>    // for std::ifstream (reading puzzle files)
#include <string>     // for std::string
#include <vector>     // for std::vector (candidate lists)
#include <cctype>     // for std::isdigit

//
==========================================================================
==
// Class SudokuGrid : owns the 9x9 board and knows how to load, print, and
```

```
//                    check legality of placements. It has no solving
logic.
//
================================================================================
==
class SudokuGrid {
public:                                          // public interface
of the class
   static const int SIZE = 9;                    // board is 9 cells
wide and tall
   static const int BOX = 3;                     // each sub-grid
(block) is 3x3

   SudokuGrid() {                                // default constructor
       for (int r = 0; r < SIZE; r++) {          // loop over every row
           for (int c = 0; c < SIZE; c++) {      // loop over every
column
               cells[r][c] = 0;                  // 0 means the cell is
empty
           }                                     // end column loop
       }                                         // end row loop
   }                                             // end constructor

   bool loadFromFile(const std::string& fileName) { // read a puzzle from
a text file
       std::ifstream in(fileName);               // open the file for
reading
       if (!in) {                                // check whether the
file opened
           return false;                         // report failure to
the caller
       }                                         // end if
       int count = 0;                            // number of cells
read so far
       char ch;                                  // the character
currently read
       while (count < SIZE * SIZE && in.get(ch)) {  // read until 81 cells
or EOF
           if (ch >= '1' && ch <= '9') {         // a digit 1-9 is a
given value
```

```
            cells[count / SIZE][count % SIZE] = ch - '0'; // store its
numeric value
            count++;                                    // move on to the next
cell
        } else if (ch == '_' || ch == '0' || ch == '.') { // blank-cell
markers
            cells[count / SIZE][count % SIZE] = 0; // store 0 for an
empty cell
            count++;                                    // move on to the next
cell
        }                                               // other chars
(spaces, newlines, |, -) are ignored
    }                                                   // end while
    return count == SIZE * SIZE;                        // true only if all 81
cells were read
}                                                       // end loadFromFile

void print() const {                                    // print the board to
the console
    for (int r = 0; r < SIZE; r++) {                    // loop over every row
        if (r % BOX == 0 && r != 0) {                   // between block
rows...
            std::cout << "------+-------+------\n"; // ...draw a
horizontal divider
        }                                               // end if
        for (int c = 0; c < SIZE; c++) {                // loop over every
column
            if (c % BOX == 0 && c != 0) {               // between block
columns...
                std::cout << "| ";                      // ...draw a vertical
divider
            }                                           // end if
            if (cells[r][c] == 0) {                     // empty cell?
                std::cout << "_ ";                      // show an underscore
            } else {                                    // filled cell
                std::cout << cells[r][c] << " "; // show its digit
            }                                           // end if/else
        }                                               // end column loop
        std::cout << "\n";                              // end the printed row
    }                                                   // end row loop
```

```
   }                                                    // end print

   int get(int r, int c) const {                        // read the value in a
cell
       return cells[r][c];                              // return it (0 =
empty)
   }                                                    // end get

   void set(int r, int c, int value) {                  // write a value into
a cell
       cells[r][c] = value;                             // store it (0 clears
the cell)
   }                                                    // end set

   bool findEmptyCell(int& row, int& col) const {       // locate the next
empty cell
       for (int r = 0; r < SIZE; r++) {                 // scan rows top to
bottom
           for (int c = 0; c < SIZE; c++) {             // scan columns left
to right
               if (cells[r][c] == 0) {                  // found an empty cell
                   row = r;                             // hand back its row
                   col = c;                             // hand back its
column
                   return true;                         // signal that one was
found
               }                                        // end if
           }                                            // end column loop
       }                                                // end row loop
       return false;                                    // no empty cells:
board is full
   }                                                    // end findEmptyCell

   bool canPlace(int r, int c, int value) const {       // is value legal at
(r, c)?
       for (int i = 0; i < SIZE; i++) {                 // check the whole row
and column
           if (cells[r][i] == value) return false;      // value already in
this row
```

```
            if (cells[i][c] == value) return false;  // value already in
this column
        }                                            // end loop
        int boxR = (r / BOX) * BOX;                  // top row of the 3x3
block
        int boxC = (c / BOX) * BOX;                  // left column of the
3x3 block
        for (int i = 0; i < BOX; i++) {              // loop over block
rows
            for (int j = 0; j < BOX; j++) {          // loop over block
columns
                if (cells[boxR + i][boxC + j] == value) return false; //
already in block
            }                                        // end inner loop
        }                                            // end outer loop
        return true;                                 // no conflicts found
    }                                                // end canPlace

    std::vector<int> candidates(int r, int c) const {// list every legal
digit for (r, c)
        std::vector<int> result;                     // container for the
legal digits
        for (int v = 1; v <= SIZE; v++) {            // try digits 1
through 9
            if (canPlace(r, c, v)) {                 // is this digit legal
here?
                result.push_back(v);                 // keep it as a
candidate
            }                                        // end if
        }                                            // end for
        return result;                               // may be empty =>
dead end
    }                                                // end candidates

    bool givensAreConsistent() const {               // verify the starting
clues don't conflict
        SudokuGrid copy = *this;                     // work on a copy of
the board
        for (int r = 0; r < SIZE; r++) {             // loop over every row
```

```
            for (int c = 0; c < SIZE; c++) {          // loop over every
column
                int v = copy.cells[r][c];             // value of this cell
                if (v != 0) {                         // only check filled
cells
                    copy.cells[r][c] = 0;             // temporarily remove
it
                    if (!copy.canPlace(r, c, v)) {    // would it conflict
with another clue?
                        return false;                 // yes: the puzzle is
invalid
                    }                                 // end inner if
                    copy.cells[r][c] = v;             // put the value back
                }                                     // end if
            }                                         // end column loop
        }                                             // end row loop
        return true;                                  // all clues are
consistent
    }                                                 // end
givensAreConsistent

private:                                               // internal data,
hidden from users
    int cells[SIZE][SIZE];                            // the 9x9 board (0 =
empty)
};                                                     // end class
SudokuGrid

//
================================================================================
==
// Class SudokuSolver : performs depth-first search with backtracking on a
//                      SudokuGrid and prints every solution it finds.
//
================================================================================
==
class SudokuSolver {
public:                                                // public interface
of the class
```

```
   explicit SudokuSolver(SudokuGrid& g)              // constructor takes
the board to solve
       : grid(g), solutionCount(0) {}                // keep a reference,
start count at 0

   int solveAll() {                                  // entry point: find
all solutions
       solutionCount = 0;                            // reset the solution
counter
       if (!grid.givensAreConsistent()) {            // contradictory clues
=> no solution
           return 0;                                 // nothing to search
       }                                             // end if
       search();                                     // start the recursive
search
       return solutionCount;                         // report how many
were found
   }                                                 // end solveAll

private:                                              // internal helpers
and data
   void search() {                                   // recursive
backtracking routine
       int row = 0, col = 0;                         // will hold the next
empty cell
       if (!grid.findEmptyCell(row, col)) {          // no empty cells =>
board complete
           solutionCount++;                          // count this solution
           std::cout << "Solution " << solutionCount << ":\n"; // label it
           grid.print();                             // print the solved
board
           std::cout << "\n";                        // blank line after
the solution
           return;                                   // back up to look for
more solutions
       }                                             // end if
       std::vector<int> options = grid.candidates(row, col); // legal
digits for this cell
       for (size_t i = 0; i < options.size(); i++) {// try each candidate
in turn
```

```
            grid.set(row, col, options[i]);          // place the candidate
            search();                                // recurse: solve the
remaining cells
            grid.set(row, col, 0);                   // backtrack: undo the
placement
        }                                            // end for (no
candidates => just returns)
    }                                                // end search

    SudokuGrid& grid;                                // the board being
solved
    int solutionCount;                               // how many solutions
found so far
};                                                    // end class
SudokuSolver

//
============================================================================
==
// Class PuzzleRunner : handles one puzzle file from loading through
reporting.
//
============================================================================
==
class PuzzleRunner {
public:                                               // public interface
of the class
    explicit PuzzleRunner(const std::string& name)   // constructor takes
the file name
        : fileName(name) {}                          // remember the file
name

    void run() {                                     // load, print, solve,
report
        std::cout << "==============================\n"; // visual
separator
        std::cout << "File: " << fileName << "\n\n";  // print the puzzle
file name
        SudokuGrid grid;                             // create an empty
board object
```

```
        if (!grid.loadFromFile(fileName)) {          // try to read the
puzzle
            std::cout << "Could not read a valid puzzle from this
file.\n\n"; // report error
            return;                                  // skip to the next
file
        }                                            // end if
        std::cout << "Puzzle:\n";                    // heading for the
original puzzle
        grid.print();                                // print the puzzle as
loaded
        std::cout << "\n";                           // blank line
        SudokuSolver solver(grid);                   // create a solver
bound to this board
        int found = solver.solveAll();               // find every solution
        if (found == 0) {                            // none found?
            std::cout << "No solution found\n\n";    // required message
        } else {                                     // at least one
solution
            std::cout << "Total solutions: " << found << "\n\n"; // summary
line
        }                                            // end if/else
    }                                                // end run

private:                                              // internal data
    std::string fileName;                            // path of the puzzle
file
};                                                   // end class
PuzzleRunner

//
============================================================================
==
// main : runs a PuzzleRunner for puzzle1.txt through puzzle5.txt
//
============================================================================
==
int main() {                                         // program entry
point
```

```
   for (int i = 1; i <= 5; i++) {                    // loop over the five
puzzle files
       PuzzleRunner runner("puzzle" + std::to_string(i) + ".txt"); //
build file name & runner
       runner.run();                                 // process that puzzle
   }                                                 // end for
   return 0;                                         // successful exit
}                                                     // end main
```

Google Gemini:

```
#include <iostream>     // Includes standard input-output stream library
for console printing
#include <fstream>      // Includes file stream library to read puzzle
text files from disk
#include <string>       // Includes string library to handle filenames and
text manipulation

class SudokuPuzzle {    // Defines the SudokuPuzzle class to encapsulate
board state and operations
private:                // Restricts access to internal data members for
object encapsulation
   char grid[9][9];    // Declares a 9x9 2D character array to store the
Sudoku grid state
   int solutionCount;  // Declares an integer counter to keep track of the
number of solutions found

public:                 // Exposes public member functions for external
interaction
   SudokuPuzzle() {    // Defines the constructor to initialize class
member variables
       solutionCount = 0; // Initializes the solution counter to zero
before loading any puzzle
       for (int r = 0; r < 9; r++) { // Loops through each row index from
0 to 8
           for (int c = 0; c < 9; c++) { // Loops through each column
index from 0 to 8
               grid[r][c] = '_'; // Sets each cell in the grid to the
blank character underscore
           } // Closes the inner loop for column initialization
```

```
        } // Closes the outer loop for row initialization
    } // Closes the default constructor body

    bool loadFromFile(const std::string& filename) { // Defines method to
load a puzzle from a file
        std::ifstream file(filename); // Opens the specified file using an
input file stream object
        if (!file.is_open()) { // Checks if the file failed to open
successfully
            return false;      // Returns false to indicate file loading
failure
        } // Closes the conditional block for file open check
        for (int r = 0; r < 9; r++) { // Loops through each row of the 9x9
grid
            for (int c = 0; c < 9; c++) { // Loops through each column of
the 9x9 grid
                file >> grid[r][c]; // Reads a single character from the
file into the grid cell
            } // Closes the inner loop for reading columns
        } // Closes the outer loop for reading rows
        file.close(); // Closes the file stream after reading all 81
characters
        solutionCount = 0; // Resets the solution counter each time a new
puzzle is loaded
        return true;   // Returns true to indicate successful file loading
    } // Closes the loadFromFile method

    void printGrid() { // Defines method to display the current state of
the Sudoku grid
        for (int r = 0; r < 9; r++) { // Loops through each row from 0 to 8
            for (int c = 0; c < 9; c++) { // Loops through each column from
0 to 8
                std::cout << grid[r][c] << " "; // Prints the character at
the cell followed by a space
            } // Closes the inner loop for printing columns
            std::cout << std::endl; // Prints a newline character at the
end of each row
        } // Closes the outer loop for printing rows
    } // Closes the printGrid method
```

```
   bool isValid(int row, int col, char num) { // Defines helper method to
check rule validity for a number
       for (int i = 0; i < 9; i++) { // Loops through all elements in the
current row and column
           if (grid[row][i] == num) { // Checks if the number already
exists in the current row
               return false;          // Returns false because duplicate
numbers violate Sudoku rules
           } // Closes the condition checking the row
           if (grid[i][col] == num) { // Checks if the number already
exists in the current column
               return false;          // Returns false because duplicate
numbers violate Sudoku rules
           } // Closes the condition checking the column
       } // Closes the loop checking rows and columns
       int startRow = row - (row % 3); // Calculates the starting row
index of the 3x3 subgrid block
       int startCol = col - (col % 3); // Calculates the starting column
index of the 3x3 subgrid block
       for (int r = 0; r < 3; r++) { // Loops through the 3 rows of the
subgrid block
           for (int c = 0; c < 3; c++) { // Loops through the 3 columns of
the subgrid block
               if (grid[startRow + r][startCol + c] == num) { // Checks if
number exists in subgrid
                   return false;                              // Returns
false if duplicate is found
               } // Closes the condition checking the 3x3 subgrid block
           } // Closes the inner loop for subgrid columns
       } // Closes the outer loop for subgrid rows
       return true; // Returns true if the placement satisfies all row,
column, and box constraints
   } // Closes the isValid method

   bool findEmptyCell(int& row, int& col) { // Defines method to locate
the next unfilled cell
       for (row = 0; row < 9; row++) { // Loops through all rows to find
an empty cell
           for (col = 0; col < 9; col++) { // Loops through all columns
within the current row
```

```
            if (grid[row][col] == '_') { // Checks if the current cell
contains an underscore
                return true;             // Returns true indicating an
empty cell was located
            } // Closes the condition checking for an empty cell marker
        } // Closes the inner loop for columns
    } // Closes the outer loop for rows
    return false; // Returns false if the entire grid has been
completely filled
} // Closes the findEmptyCell method

void solve(int row = 0, int col = 0) { // Defines recursive
backtracking search method
    int nextRow, nextCol; // Declares variables to hold coordinates of
the next empty cell
    if (!findEmptyCell(nextRow, nextCol)) { // Checks if there are no
more empty cells left
        solutionCount++; // Increments the total count of valid
solutions discovered
        std::cout << "Solution " << solutionCount << ":" << std::endl;
// Prints solution header
        printGrid(); // Calls printGrid to output the completed
solution configuration
        std::cout << "------------------------" << std::endl; // Prints
a separator line
        return; // Returns from current recursion branch to continue
searching for more solutions
    } // Closes the condition handling a fully solved board
    for (char digit = '1'; digit <= '9'; digit++) { // Iterates through
candidates '1' through '9'
        if (isValid(nextRow, nextCol, digit)) { // Checks if candidate
digit is valid in current cell
            grid[nextRow][nextCol] = digit; // Places valid candidate
digit into the empty cell
            solve(nextRow, nextCol); // Recursively calls solve to
progress to the next empty cell
            grid[nextRow][nextCol] = '_'; // Backtracks by resetting
the cell back to underscore
        } // Closes the condition checking candidate validity
    } // Closes the loop iterating through candidate digits 1 through 9
```

```
    } // Closes the solve method

    int getSolutionCount() { // Defines getter method to retrieve the total
number of solutions found
        return solutionCount; // Returns the stored solution count value
    } // Closes the getSolutionCount method
}; // Closes the SudokuPuzzle class definition

int main() { // Defines the main entry point function of the program
    std::string filenames[5] = {"puzzle1.txt", "puzzle2.txt",
"puzzle3.txt", "puzzle4.txt", "puzzle5.txt"}; // Array of puzzle filenames
    for (int i = 0; i < 5; i++) { // Loops through each of the five puzzle
files
        SudokuPuzzle myPuzzle; // Instantiates a SudokuPuzzle object for
the current puzzle file
        std::cout << "========================" << std::endl; // Prints
visual separator block
        std::cout << "Processing file: " << filenames[i] << std::endl; //
Prints current filename
        if (!myPuzzle.loadFromFile(filenames[i])) { // Attempts to load
puzzle data from disk file
            std::cout << "Error: Could not open file " << filenames[i] <<
std::endl; // Prints error message if file loading fails
            continue; // Skips to the next file iteration in the loop
        } // Closes the error-handling conditional block
        std::cout << "Original Puzzle:" << std::endl; // Prints header for
the original puzzle state
        myPuzzle.printGrid(); // Calls printGrid method to display the
unsolved input puzzle
        std::cout << "------------------------" << std::endl; // Prints
visual separator divider line
        myPuzzle.solve(); // Calls the recursive solve method to compute
and print all solutions
        if (myPuzzle.getSolutionCount() == 0) { // Checks if zero solutions
were found after search
            std::cout << "No solution found" << std::endl; // Prints
message stating no solution exists
        } // Closes condition checking for zero solutions
        std::cout << "========================" << std::endl << std::endl;
// Prints ending block separators
```

```
    } // Closes the loop processing all five puzzle files
    return 0; // Returns exit code 0 to indicate successful program
execution
} // Closes the main function body
```

e. Analyze the code in terms of correctness, execution time, space complexity, and maintainability as explained in the "GenAI Code Comparison" lecture.

Correctness :

The raw code produced by Gemini performed exactly as expected on the first run. The program produced the correct number of solutions for the 5 test puzzles given. It also included code to handle if the wrong files were given or if not enough files were given. The output did not include the number of solutions at the bottom of each puzzle section , but each solution given for a puzzle was labled with a number. The Claude program also runs exactly as expected. It has a more structured output format to mimic the grid of a typical sudoku puzzle. It aalso prints the total number of solutions at the bottom for each puzzle.

Execution Time:

Both programs have approximately the same worst-case execution time of O(9^k) because both use recursive depth-first search with backtracking, where k represents the number of empty cells. The Gemini program may execute slightly faster for valid puzzles because it directly tests the digits 1–9 instead of creating a vector of candidates. The Claude program performs additional work by checking the starting puzzle for conflicts and creating candidate lists. However, its initial validity check could improve execution time for an invalid puzzle by preventing an unnecessary recursive search. Overall, the difference in execution time between the programs is small because the recursive backtracking search is the main factor affecting performance.

Space Complexity:

Both programs have an auxiliary space complexity of O(k), where k is the number of empty cells in the Sudoku puzzle. This is mainly caused by the recursive call stack used during backtracking. The 9 × 9 Sudoku grid itself requires constant space because its size never changes. Claude uses slightly more memory because each recursive call creates a vector containing possible candidates, while Gemini directly tests the digits 1 through 9 without storing a candidate list. Claude also creates a temporary copy of the Sudoku grid when checking whether the starting clues are valid. Therefore, Gemini is slightly more space-efficient, although both programs have the same overall O(k) space complexity.

Maintainability – ChatGPT:
Both programs have significant comments. There are comments on every line. The Claude program includes comments that are more clear and concise then the Gemini comments. The Claude program is also organized into sections making it easier to follow.

f. Select one to use as the basis for this assignment and justify why you picked it.

I am going to use the Claude generated code as the basis for this assignment. Both programs are generally on par with each other with the same error handling. Space and Time complexity are very similar between the two programs. The formatting in thee Claude Program is closer to the assignment instructions and more visually appealing. The code is also easier to follow compared to the Gemini generated code.

g. Explain what you did to improve it in terms of correctness, execution time, space complexity, and maintainability.

I added prologue comments and edited the comments on every line, butchanged othing that would affect the running of the program. It functions as expected and prints the desired output at a desirable speed and with minimal space requirements.